Python Programming

Shaik Abdul Gaffar, MCA
Business & Management Department
Yogi Vemana University

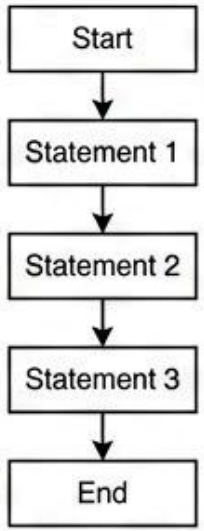
Control Flow Statements in Python

Definition

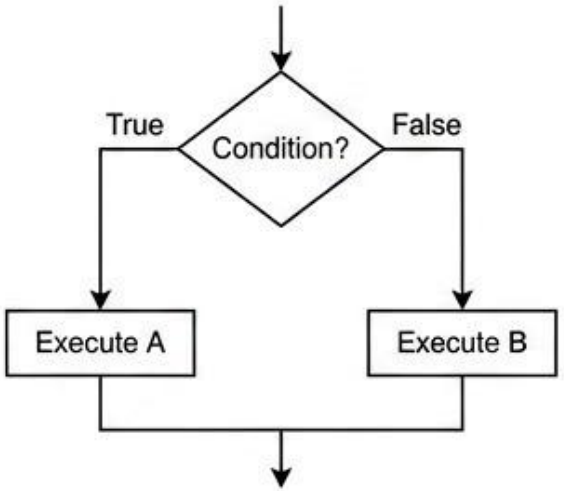
Control flow refers to the order in which statements within a program execute. execute. Normally, programs follow a **sequential flow** from top to bottom to bottom. However, sometimes we need more flexibility in execution.

They allow:

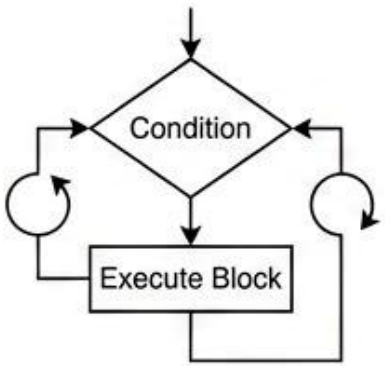
- Executing a block **multiple times**
- Executing code based on **conditions**
- **Terminate** or **skip** certain statements



Sequential Execution



Decision Making (if / elif / else)

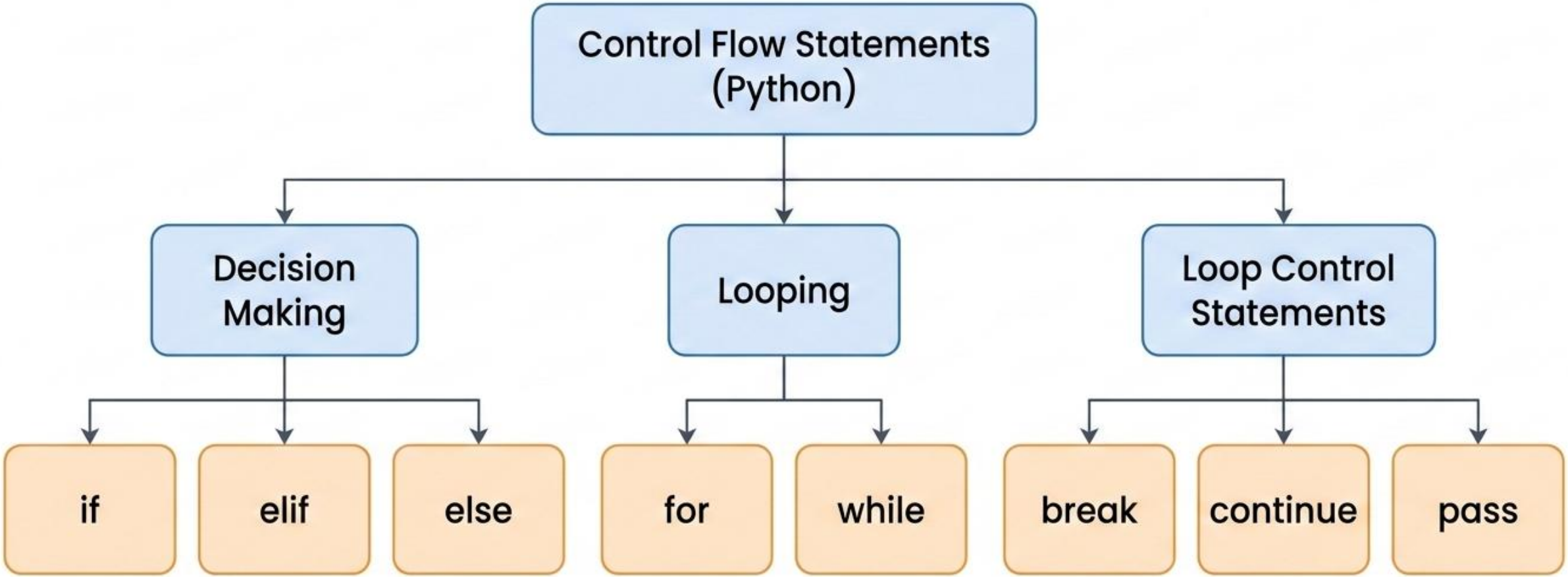


Looping (for / while)

```
for i in range(5):
    if i == 3:
        break
```

- **break** → stops loop
- **continue** → skips current iteration
- **pass** → does nothing

Loop Control Example

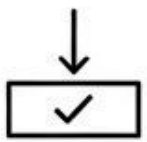

Control Flow Statements (Python)
Decision Making
Looping
Loop Control Statements
if
elif
else
for
while
break
continue
pass

Decision-Making Statements in Python

Decision-making statements help a program make **logical choices** during execution. They evaluate a specific condition to determine whether it is **True** or **False**, and then execute the corresponding block of code based on that result.

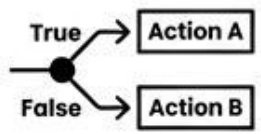
If a condition is true, perform one action; otherwise, perform a different action.

A. if



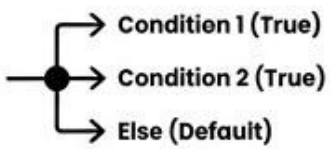
Executes a block only when the condition is true.

B. if-else



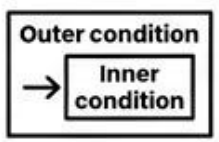
Provides two alternative actions based on the condition.

C. if-elif-else



Checks multiple conditions and executes the first true one.

D. Nested if



An if statement placed inside another if statement.

Python If Statement

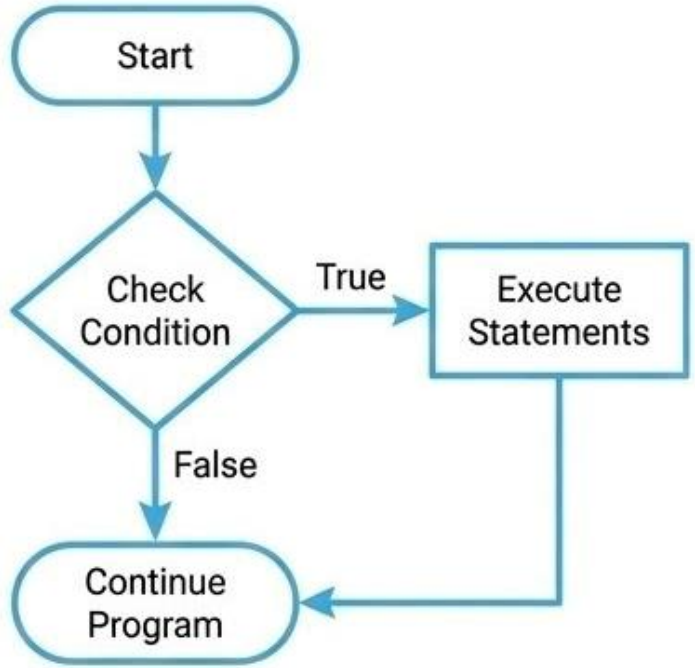
What is an If Statement?

The if statement is a decision-making construct in Python. It checks a condition and executes a block of statements only if the condition is true. If the condition is false, the program skips the block and continues execution.

Syntax

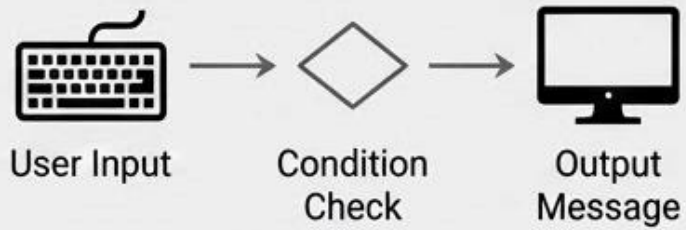
```
if condition:
    statement1
    statement2
    ...
```

The statements inside the block are executed only when the condition evaluates to True.



Example

```
age = int(input("Enter Age: "))

if age >= 18:
    print("You are eligible for vote")

if age < 0:
    print("You entered Negative Number")
```



Python if-else Statement

What is an if-else Statement?

The if-else statement is used for decision making in Python.

It evaluates a condition and executes one of two blocks of code.

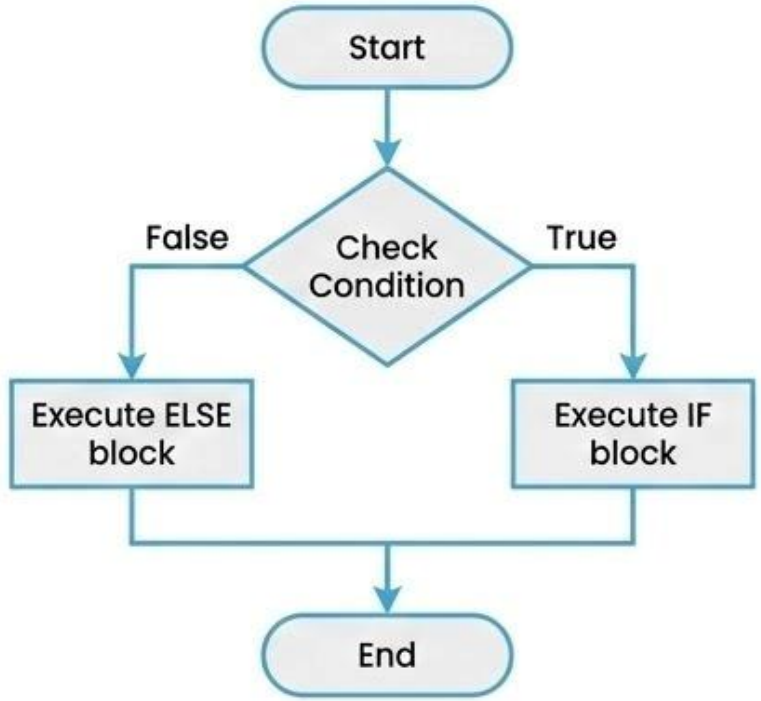
If the condition is True, the if block is executed. If the condition is False, the else block is executed.

This allows a program to take different actions based on conditions.

Syntax

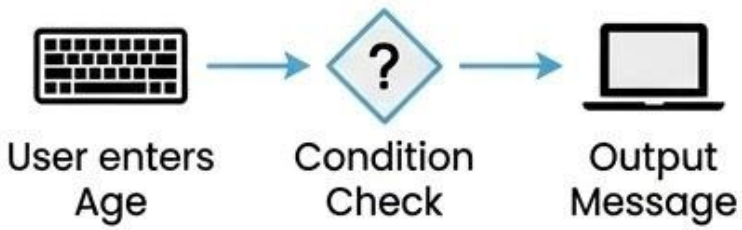
```
if condition:
    statement
else:
    statement
```

Only one block executes depending on whether the condition is True or False.



Example

```
age = int(input("Enter Age: "))

if age >= 18:
    print("You are eligible for vote")
else:
    print("You are not eligible for vote")
```



"Python Ladder if-elif-else Statement"

What is an if-elif-else Statement?

The if-elif-else statement is a multi-decision making construct in Python.

It allows a program to check multiple conditions sequentially.

If the first condition is True, its block is executed.

If it is False, Python checks the next condition using elif.

If none of the conditions are true, the else block executes.

This structure is called an if-elif-else ladder.

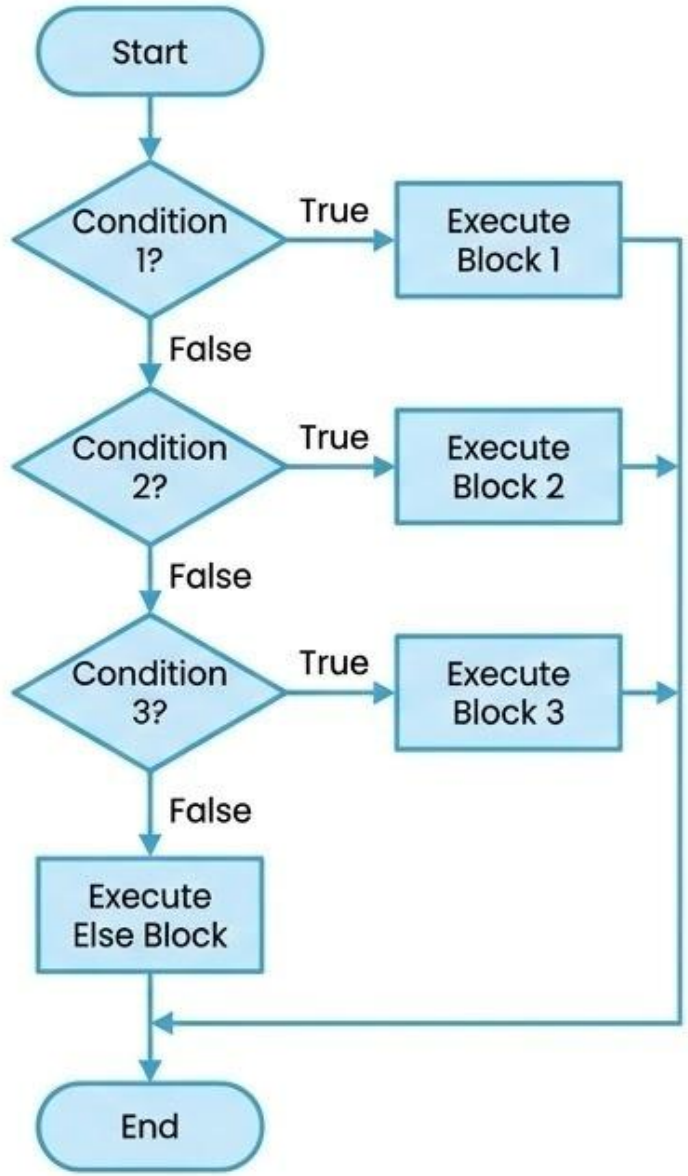
Syntax

```
if condition1:
    statement1
    statement2
elif condition2:
    statement1
    statement2
elif condition3:
    statement1
    statement2
else:
    statement1
    statement2
```

Conditions are checked from top to bottom, and only the first true block is executed.

Example

```
num = int(input("Enter Number: "))

if num > 0:
    print("You entered positive number")
elif num < 0:
    print("You entered negative number")
else:
    print("You entered zero")
```



"Python Nested if-else Statements"

What are Nested if-else Statements?

A nested if statement is an if-else statement placed inside another if or else block.

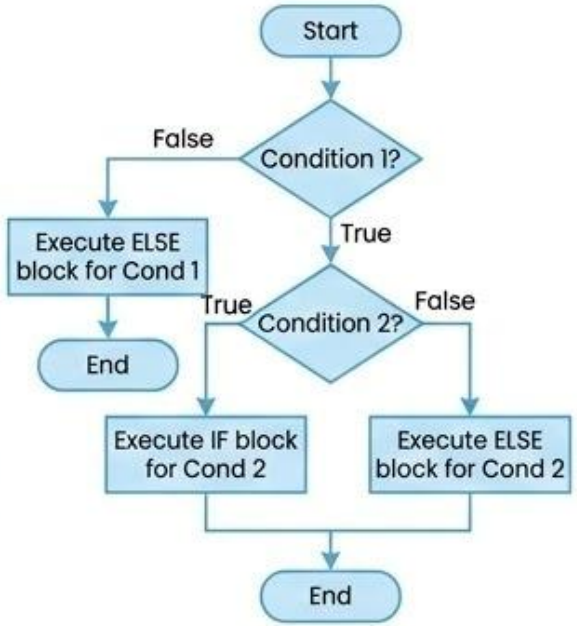
It allows a program to check for further conditions only when an outer condition is met.

This creates a hierarchy of decision-making, enabling more complex logic within a single structure.

Syntax

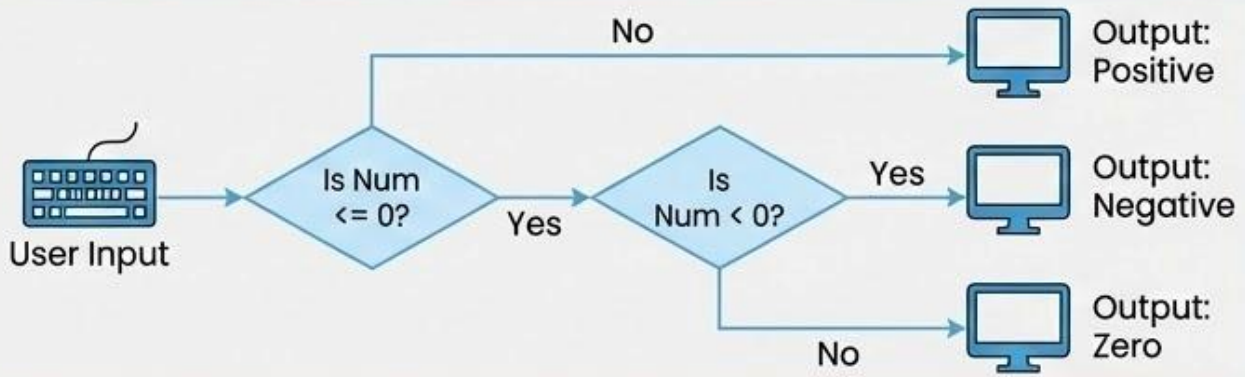
```
if condition1:
    if condition2:
        # Executes if condition1 AND condition2 are True
        statement(s)
    else:
        # Executes if condition1 is True, but condition2 is False
        statement(s)
else:
    # Executes if condition1 is False
    statement(s)
```

Inner blocks are indented to show their relationship to the outer blocks.



Example

```
num = int(input("Enter Number: "))

if num <= 0:
    if num < 0:
        print("You entered Negative number")
    else:
        print("You entered Zero")
else:
    print("You entered Positive number")
```



"Python Example: Finding the Largest of Three Numbers"

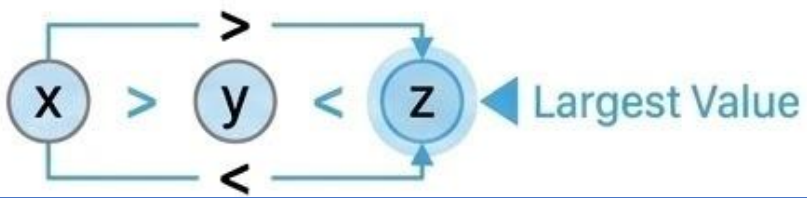
Problem

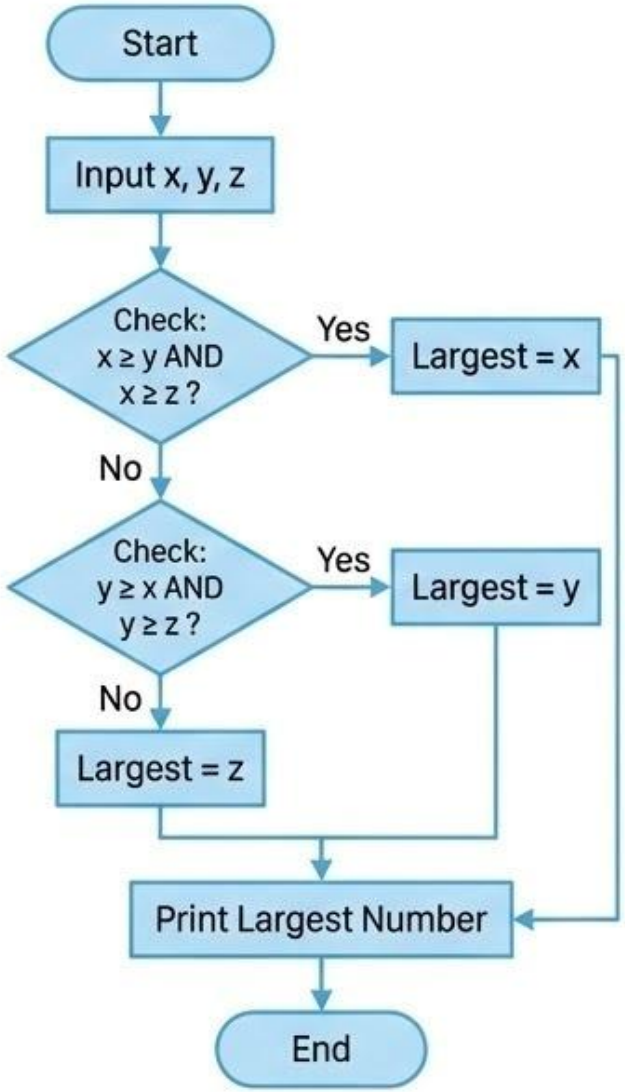
Write a Python program to find the largest number among three input numbers.

The program uses if-elif-else conditional statements to compare compare the values and determine the largest number.

Python Program

```
x = int(input("Enter First Number: "))
y = int(input("Enter Second Number: "))
z = int(input("Enter Third Number: "))

if (x >= y and x >= z):
    largest = x
elif (y >= x and y >= z):
    largest = y
else:
    largest = z

print(f"The largest number is: {largest}")
```





“Python Iteration Statements (Loops)”

What are Iteration Statements?

Iteration statements (loops) are used to execute a block of statements repeatedly based on a condition.

The repetition continues as long as the condition evaluates to True.

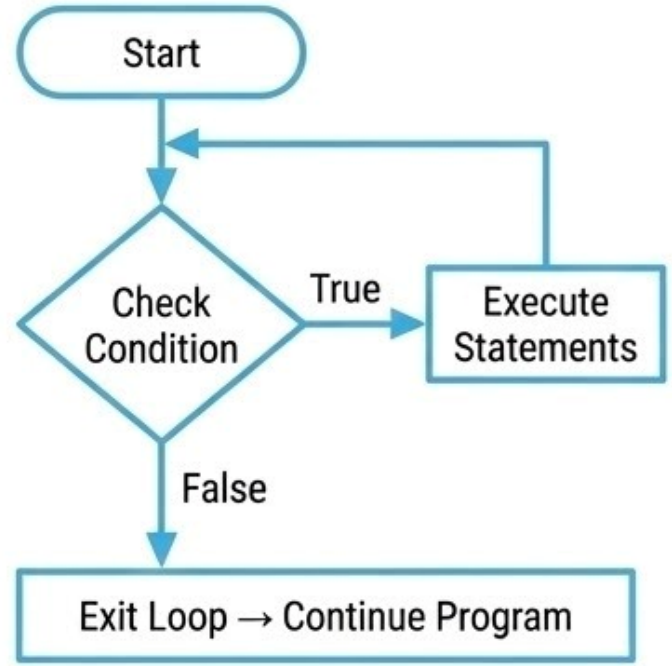
When the condition becomes False, the loop stops and the program continues with the next statement.

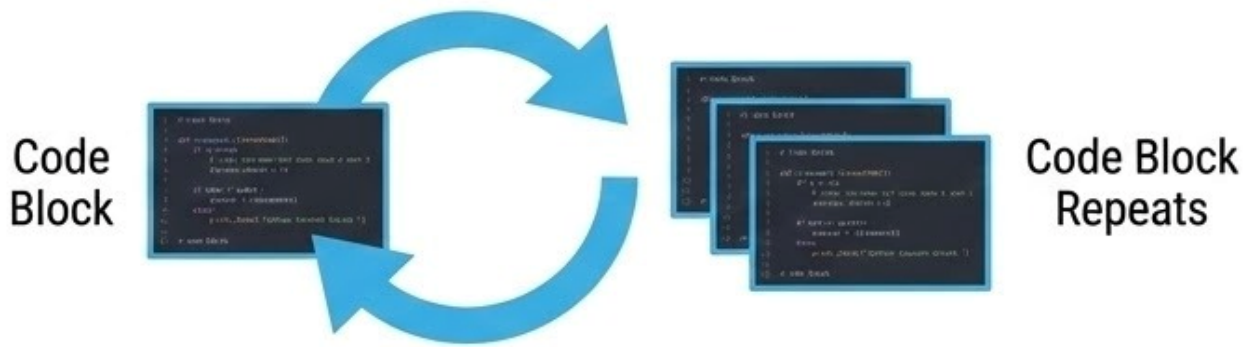
Loops help automate tasks that require repeated execution of code.

Types of Loops in Python

- while loop
- for loop

Both loops allow programmers to repeat instructions efficiently without writing the same code multiple times.





“Loops repeat instructions automatically until the condition becomes false.”

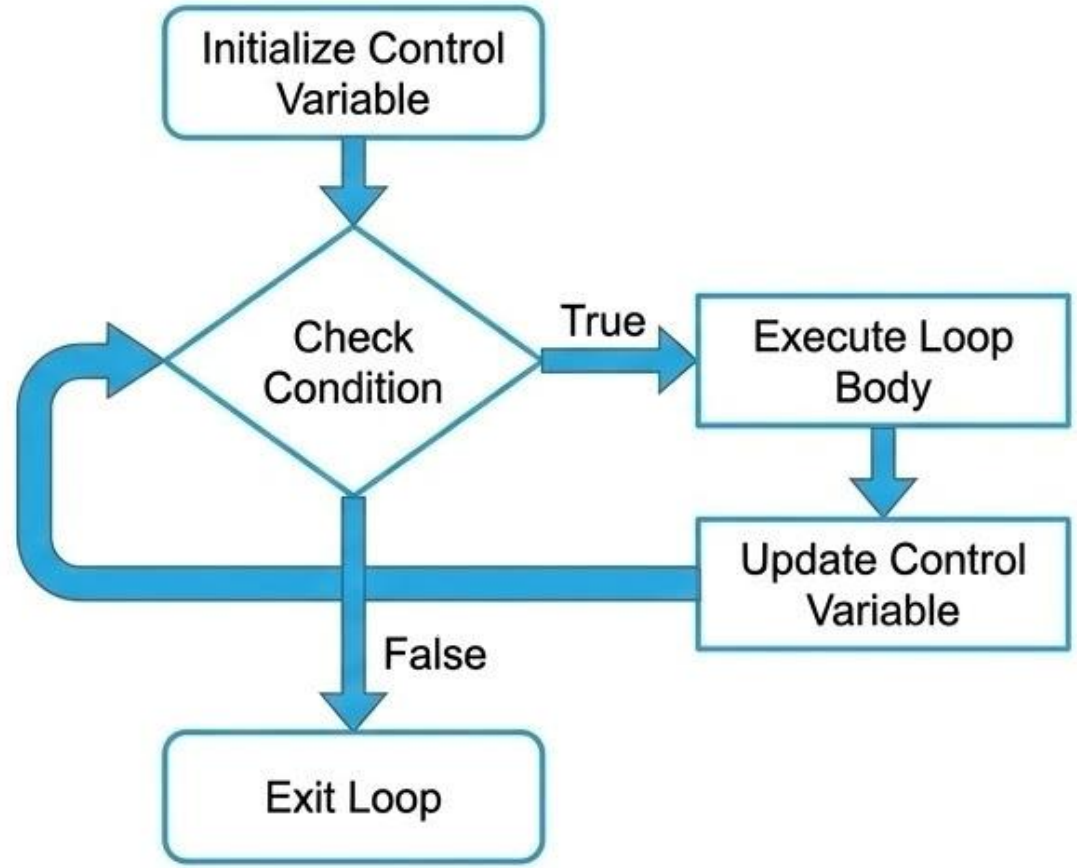
“Four Essential Parts of a Loop”

Understanding Loop Components

A loop in Python has four essential parts that control its execution:

- Initialization of control variable – Set the starting value.
- Condition testing – Check if the loop should continue based on the control variable.
- Body of loop – Statements that are executed repeatedly.
- Increment/Decrement – Update the control variable to eventually terminate the loop.

These four components ensure proper loop execution and prevent infinite loops.



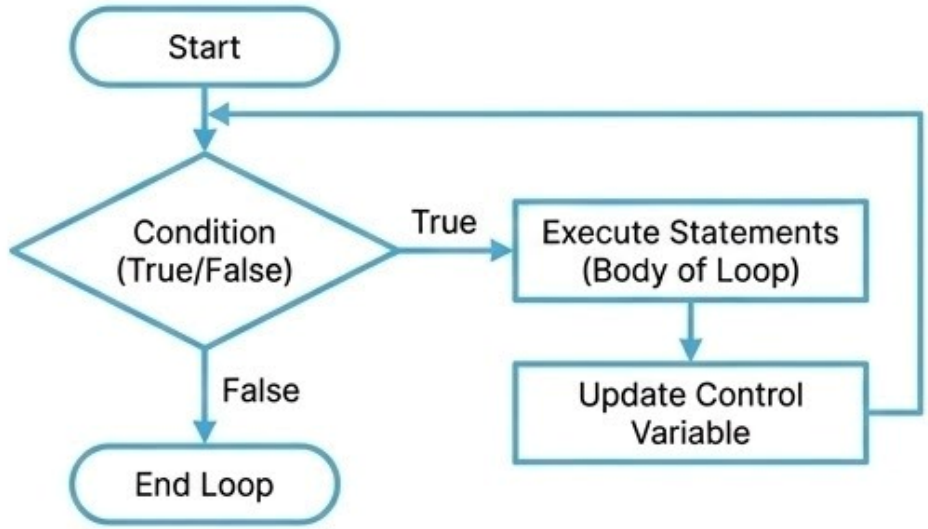
Key Idea: All loops require initialization, condition, body, and update to function correctly and terminate as expected.

Python while Loop

What is a while loop?

The while loop is a conditional construct that executes a block of statements again and again till the given condition remains true. Whenever the condition meets a result of false, then the loop will terminate.

Flowchart



Syntax

```
Initialization of control variable
while (condition):
    statement
    Updation in control variable
.................
```

Example: Print 1 to 10

```
num = 1                     # initialization
while(num <= 10):           # condition testing
    print(num, end=' ') # Body of loop
    num += 1                # Increment
```

Python Example: Sum of Numbers from 1 to 10

Problem

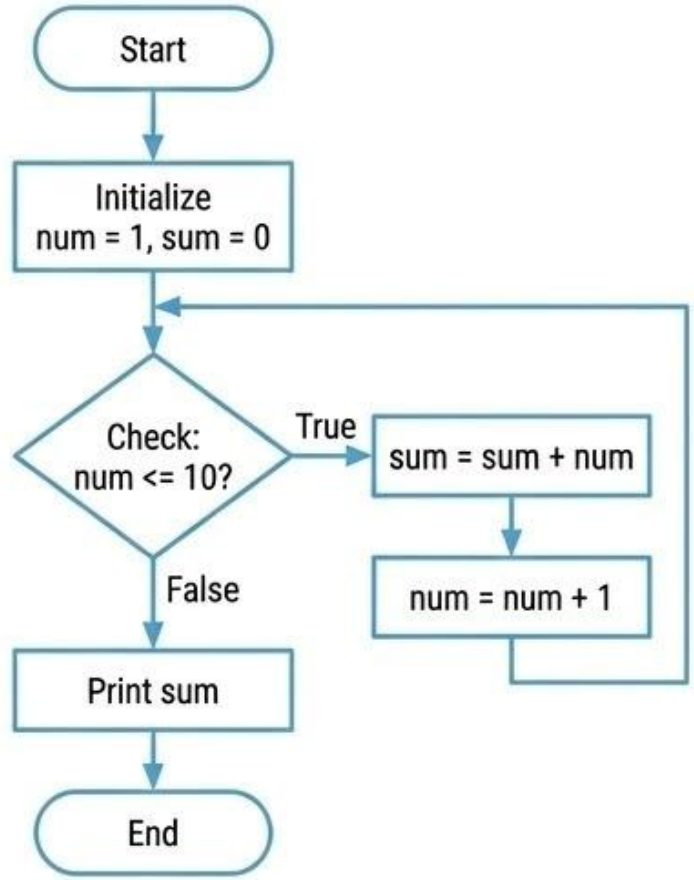
Write a Python program to calculate the sum of numbers from 1 to 10 using a while loop. The program will:

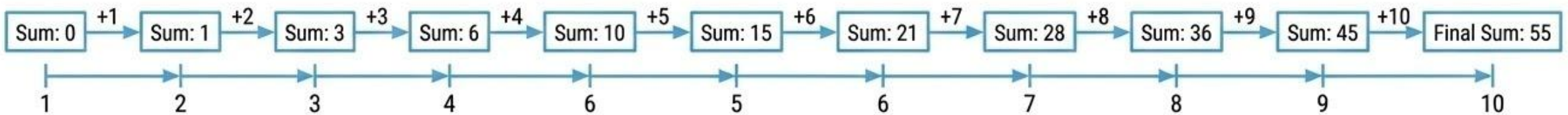
- Initialize a control variable (num)
- Keep adding numbers to sum while the condition is True
- Increment the control variable in each iteration
- Print the final sum

Python Program

```
num = 1
sum = 0

while num <= 10:
    sum += num
    num += 1

print("The Sum of 1-10
numbers:", sum)
```

sum += num adds the current number to the total sum
num += 1 increments the control variable





Python range() Function

What is range()?

The **range()** function in Python returns a sequence of numbers.

By default:
- Starts at 0
- Increments by 1
- Ends before the specified stop value.

The general format of **range():**
range(start, stop, step)

Where:
start → starting number (optional, default = 0),
stop → end number (exclusive),
step → increment/decrement (optional, default = 1)

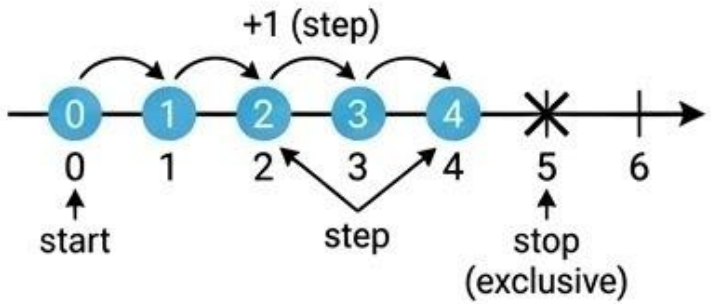
Parameters Explanation

Parameter	Description	Default
start	Lower limit of the sequence	0
stop	Upper limit of the sequence (not included)	Required
step	Increment / Decrement	1

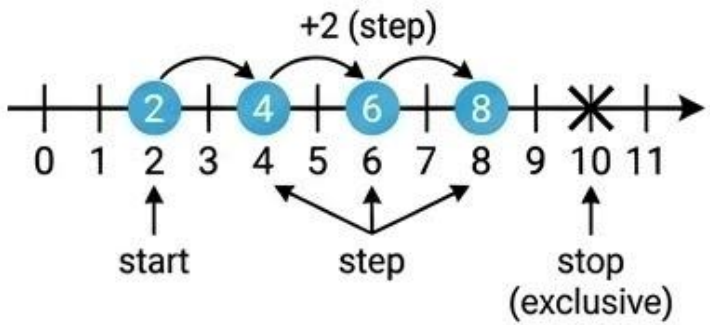
Note: The start and step parameters are optional, while stop is mandatory.

Visual Illustration

range(0, 5) → highlights numbers:



range(2, 10, 2) → numbers: (Optional)



Key Idea: range() is widely used in Python for looping and generating sequences of numbers efficiently.

Python range() Function Examples

Examples of range()

range(5) → 0, 1, 2, 3, 4

range(2, 5) → 2, 3, 4

range(1, 10, 2) → 1, 3, 5, 7, 9

range(5, 0, -1) → 5, 4, 3, 2, 1

range(0, -5) → [] (empty list, default step = +1)

range(0, -5, -1) → 0, -1, -2, -3, -4

range(-5, 0, 1) → -5, -4, -3, -2, -1

range(-5, 1, 1) → -5, -4, -3, -2, -1, 0

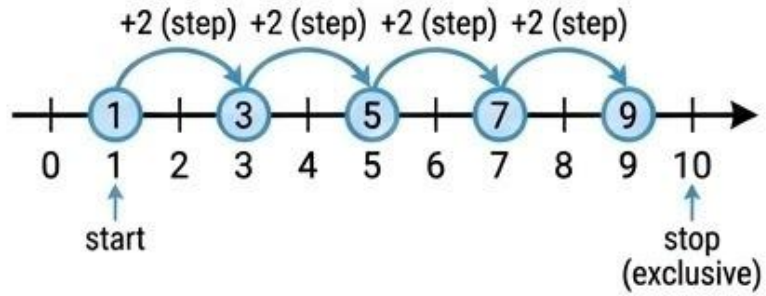
Using range() with list()

```
L = list(range(1, 20, 2))
print(L)
# Output: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

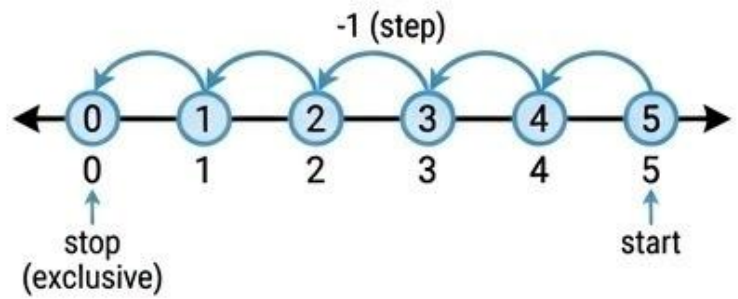
list(range(...)) converts the range into a Python list for easier visualization.

Visual Illustration

range(1, 10, 2)



range(5, 0, -1)



Key Idea: range() can generate sequences in ascending or descending order and can skip numbers using the step parameter. It is commonly used in loops and list generation.

Python for Loop

What is a for Loop?

The for loop is used to iterate over a sequence such as a list, tuple, string, or range.

Executes a block of statements once for each element in the sequence.

Commonly used for counting, traversing lists, or performing repeated operations.

Syntax & Examples

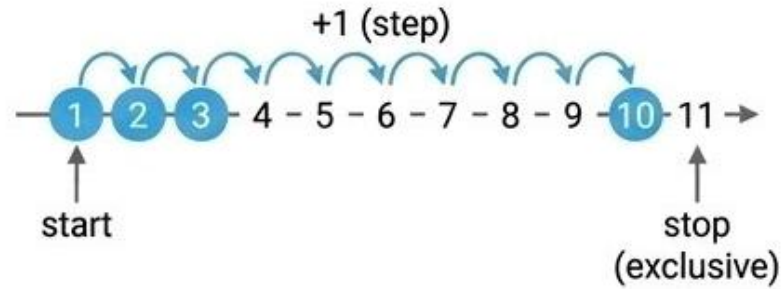
```
# Example 1: Print numbers 1 to 10
for num in range(1, 11, 1):
    print(num, end=" ")
# Output: 1 2 3 4 5 6 7 8 9 10

# Example 2: Print numbers 10 to 1
for num in range(10, 0, -1):
    print(num, end=" ")
# Output: 10 9 8 7 6 5 4 3 2 1
```

Note:

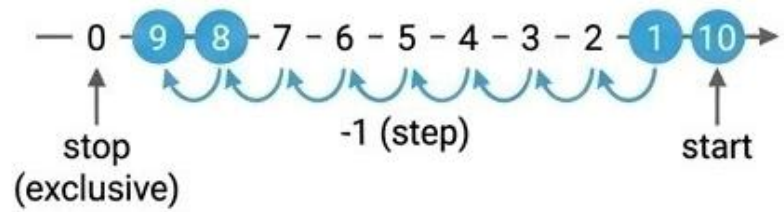
- range(start, stop, step) controls iteration
- end=" " prints numbers on the same line

Visual Illustration

Example 1: 1 → 2 → 3 → ... → 10



Example 2: 10 → 9 → 8 → ... → 1



Key Idea: The for loop iterates over sequences efficiently, making it ideal for counting, processing lists, and repetitive tasks.

Python for Loop: Iterating Over Sequences

Iterating Over Lists and Strings

The for loop can iterate over any sequence, such as:

Lists – to access each element

Strings – to access each character

Each element is stored in a **loop variable** during each iteration.

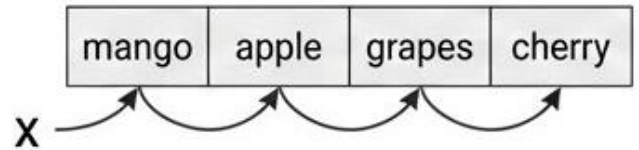
Examples

```
Example 1: Iterate over a list
fruits = ["mango", "apple", "grapes", "cherry"]
for x in fruits:
    print(x)
Output:
mango
apple
grapes
cherry

Example 2: Iterate over a string
for x in "TIGER":
    print(x)
Output:
T
I
G
E
R
```

The loop variable x takes the value of each element/character in sequence.

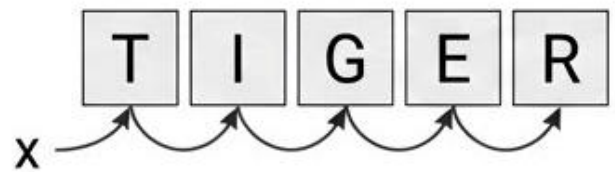
Visual Illustration

List example:
fruits = ["mango", "apple", "grapes", "cherry"]



Loop Variable → Current Element

String example: "TIGER"



Loop Variable → Current Element

Key Idea: The for loop can iterate over any sequence—lists, strings, tuples, or ranges—allowing easy access to each element.

Python for Loop: else & Nested Loops

Using else with for Loop

The else block in a for loop executes after the loop finishes normally (without a break).

```
for x in range(4):
    print(x, end=" ")
else:
    print("\nFinally finished!")
```

Output:

```
0 1 2 3
Finally finished!
```

Note: else is executed only after the loop completes all iterations.

Nested Loops (Loop inside a Loop)

A nested loop is a loop inside another loop. It allows iteration over multiple sequences or multi-dimensional data.

```
city = ["Jaipur", "Delhi", "Mumbai"]
fruits = ["apple", "mango", "cherry"]

for x in city:
    for y in fruits:
        print(x, ":", y)
```

Output:

```
Jaipur : apple
Jaipur : mango
Jaipur : cherry
Delhi  : apple
...
Mumbai : cherry
```

Note: Outer loop picks a city. Inner loop iterates over fruits for each city.

Visual Illustration

Python Loop Control Statements

What are Loop Control Statements?

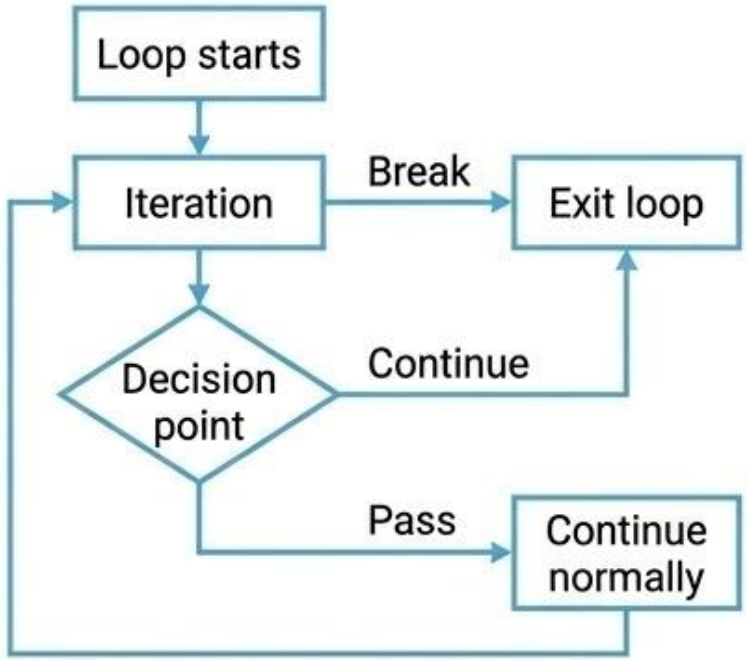
Loop control statements in Python are special statements that help manage the execution of loops (for or while).

They allow you to modify the default behavior of a loop:
Stop it early
Skip an iteration
Do nothing temporarily

Python Supports Three Control Statements

- **Break** statement – exits the loop immediately
- **Continue** statement – skips the current iteration and moves to the next
- **Pass** statement – acts as a placeholder; does nothing

Visual Illustration



Key Idea: Loop control statements provide flexibility and control over loop execution
Key Idea: Loop control statements provide flexibility and control over loop execution without modifying the loop structure.

Python break Statement

What is break?

The break statement in Python is used to exit a loop prematurely, whether it is a for loop or while loop.

Key points:

- Stops the loop immediately
- Control moves to the next statement after the loop
- Useful to end iteration early when a condition is met

Examples

For Loop Example:

```
for i in range(5):
    if i == 3:
        break  # Exit the loop
    print(i)

# Output:
0
1
2
```

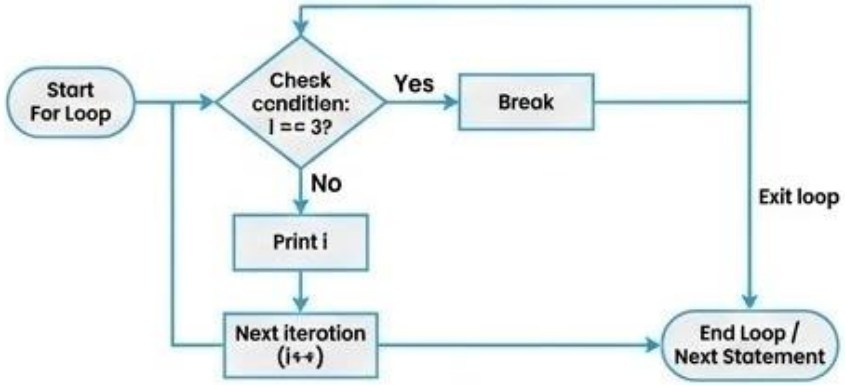
While Loop Example:

```
i = 0
while i < 5:
    if i == 3:
        break  # Exit the loop
    print(i)
    i += 1

# Output:
0
1
2
```

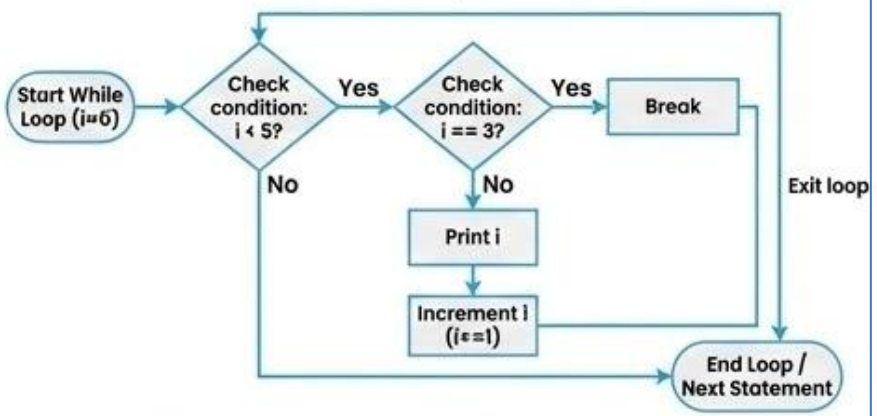
Note: break prevents further iteration once the condition is True

Visual Illustration

For Loop Flow



While Loop Flow



Key Idea: The break statement allows immediate termination of loops, making your program efficient by avoiding unnecessary iterations.

Python continue Statement

What is continue?

The continue statement in Python is a loop control statement that skips the rest of the code inside the loop for the current iteration and proceeds to the next iteration.

Key points:

- Only affects the current iteration
- The loop continues running for remaining elements
- Useful to skip certain conditions without terminating the loop

Examples

For Loop Example:

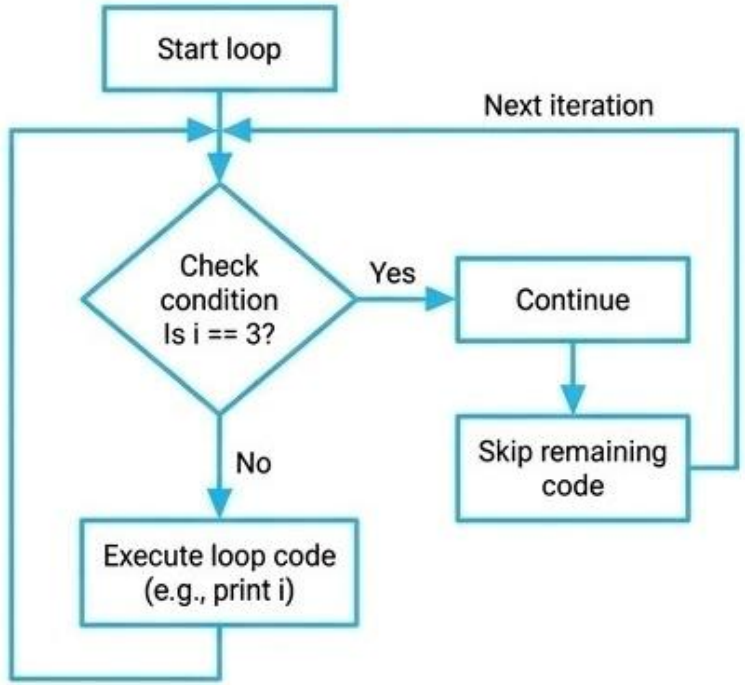
```
for i in range(5):
    if i == 3:
        continue  # Skip iteration when i = 3
    print(i)

# Output:
0
1
2
4
```

While Loop Example:

```
count = 0
while count < 6:
    count += 1
    if count == 3:
        continue # Skip iteration when count = 3
    print(count)

# Output:
1
2
4
5
6
```

Note: continue skips only the current iteration, unlike break which stops the loop entirely

Visual Illustration



Key Idea: The **continue** statement allows selective skipping of loop iterations without exiting the loop, making your program more flexible and efficient.

Python pass Statement

What is pass?

The **pass** statement in Python is a null operation or placeholder.

- Does nothing when executed
- Used when a statement is syntactically required but you don't want any code to run
- Helps maintain program structure

Examples

For Loop Example:

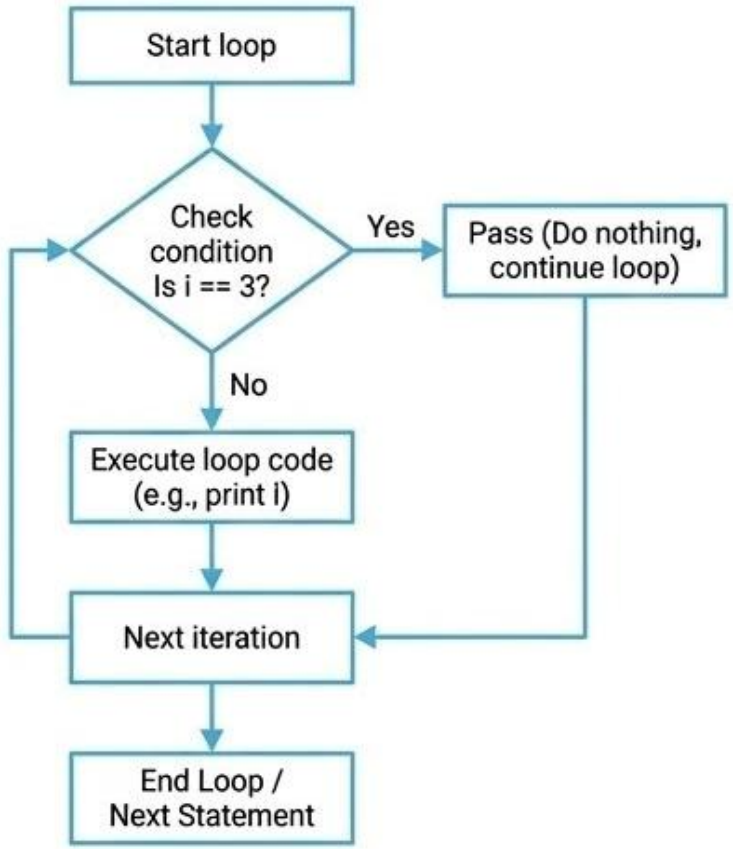
```
for i in range(5):
    if i == 3:
        pass  # Placeholder for future code
    print(i)

# Output:
0
1
2
3
4
```

While Loop Example:

```
count = 0
while count < 5:
    count += 1
    if count == 3:
        pass # Placeholder for future code; does nothing
    print(f"Current count: {count}")

# Output:
Current count: 1
Current count: 2
Current count: 3
Current count: 4
Current count: 5
```

Note: pass does not affect loop execution; it is only a syntactic placeholder

Visual Illustration

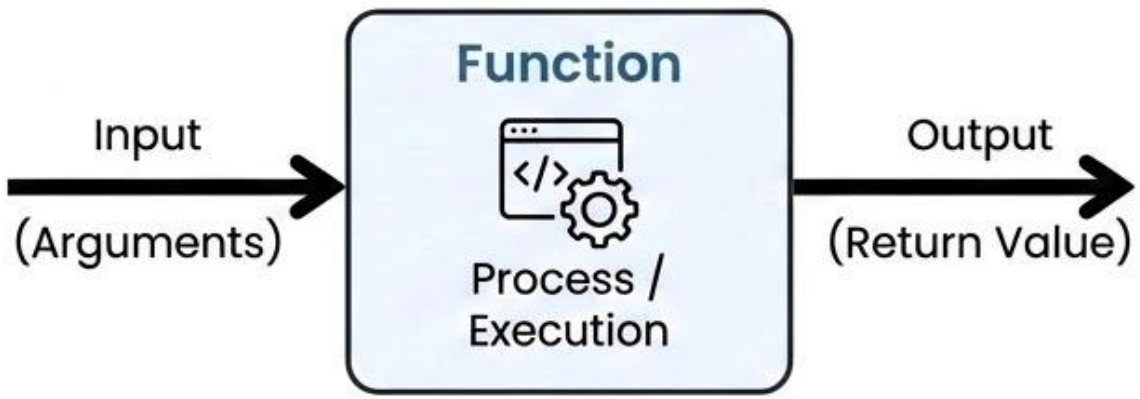


Key Idea: The pass statement is a placeholder that allows Python programs to maintain structure without executing code, useful during development.

Functions in Python

What is a Function?

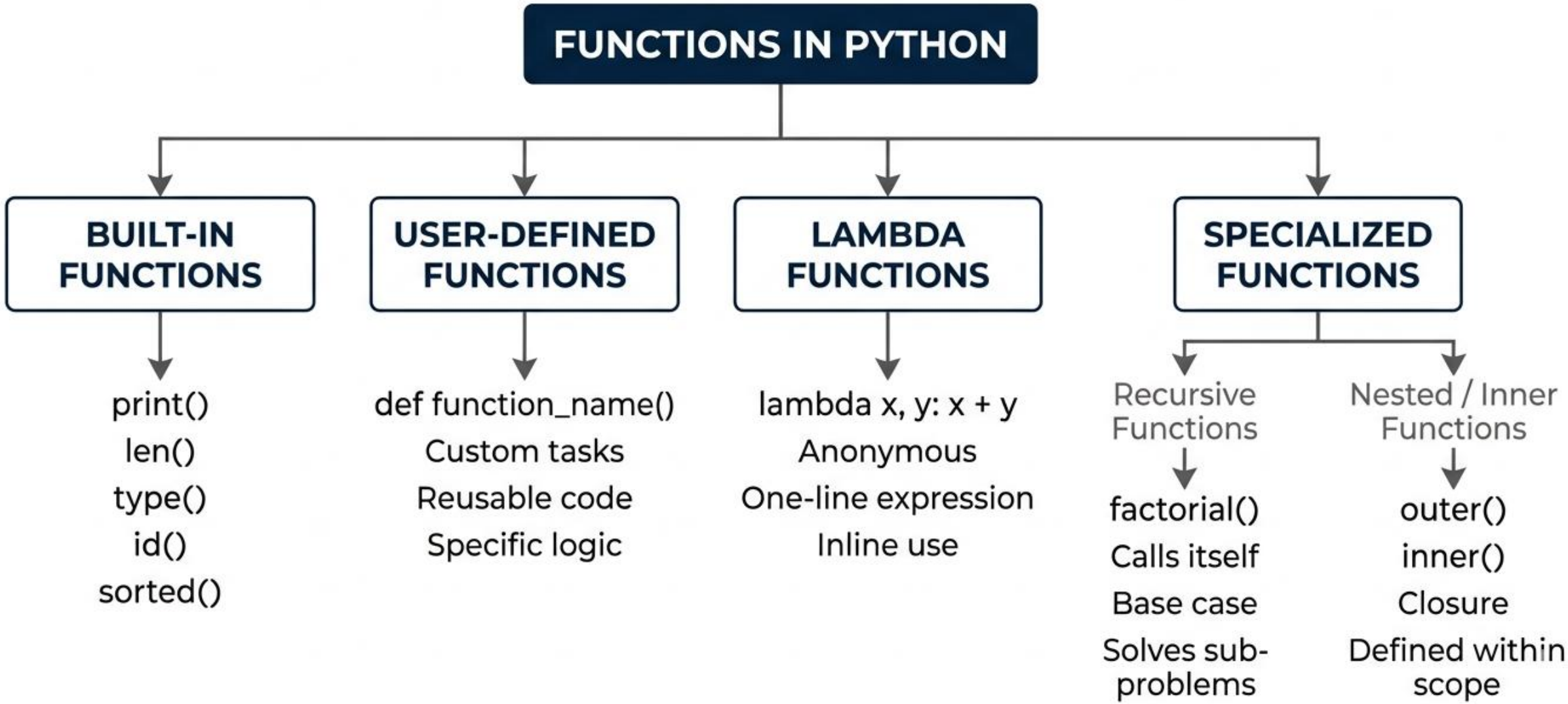
- A reusable block of code that performs a specific task
- Executes only when it is called
- Helps avoid repetition (DRY principle)
- Allows reuse of the same code with different inputs



Why Use Functions?

- Code Reusability
- Modularity
- Readability
- Maintainability

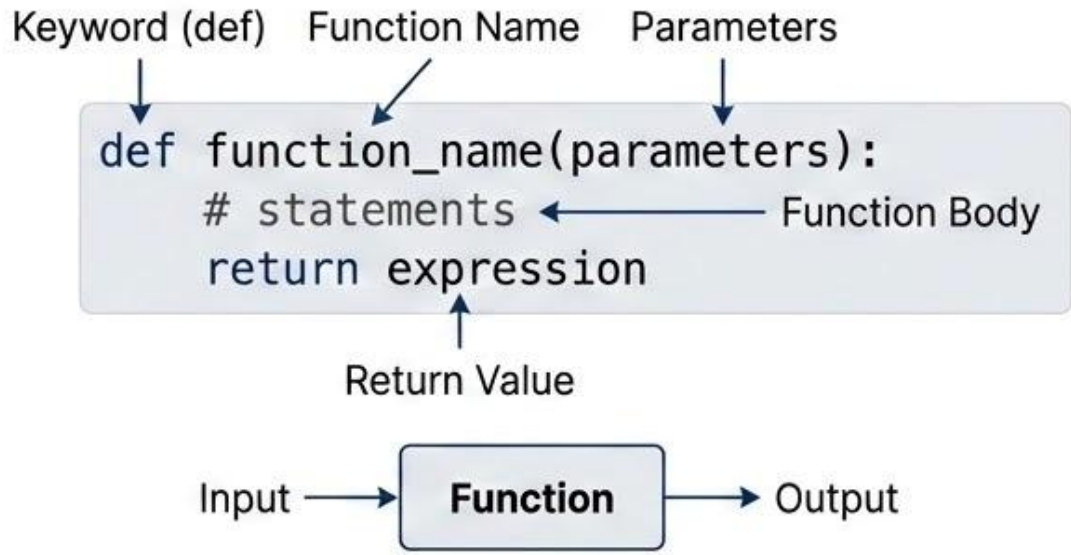
TYPES OF FUNCTIONS IN PYTHON



Defining and Calling a Function in Python

Defining a Function

- Use the def keyword to define a function
- A function can take parameters as input
- Contains statements inside the function body
- May return a value using return



Calling a Function

- Call a function using its name
- Use parentheses () after the function name
- Executes the function code

```
def yvumba():
    print("Welcome to YVU MBA")

yvumba()
```

Output:
Welcome to YVU MBA

Return Statement in Python Functions

Concept + Syntax

What is return?

- Ends the execution of a function
- Sends a **value** back to the caller
- Can **return** any data type
- Can **return** multiple values (as a tuple)
- **Returns** None if no value is specified

Syntax

```
return [expression]
```

Example + Output

Example

```
def sq_value(num):
    """Returns square of number"""
    return num**2

print(sq_value(2))
print(sq_value(-4))
```

```
4
16
```

return sends the result back instead of just printing it

Assigning Functions to Variables in Python

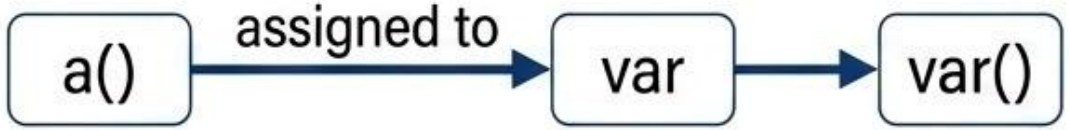
Concept Explanation

Functions as First-Class Objects

- Functions can be **assigned** to variables
- Functions can be **passed** as **arguments**
- Functions can be **returned** from other functions
- Enables **flexible** and **reusable** code

Key Idea

A function can be called using a **variable name.**



Example + Output

Example

```
def a():
    print("GFG")

var = a

var()
```

Output

```
GFG
```

"**var** refers to the **function**, not the result of the function"

Function Arguments in Python

What are Arguments?

- **Arguments** are values passed inside function parentheses.
- Used to provide input to functions.
- A function can have multiple arguments separated by commas.

Syntax

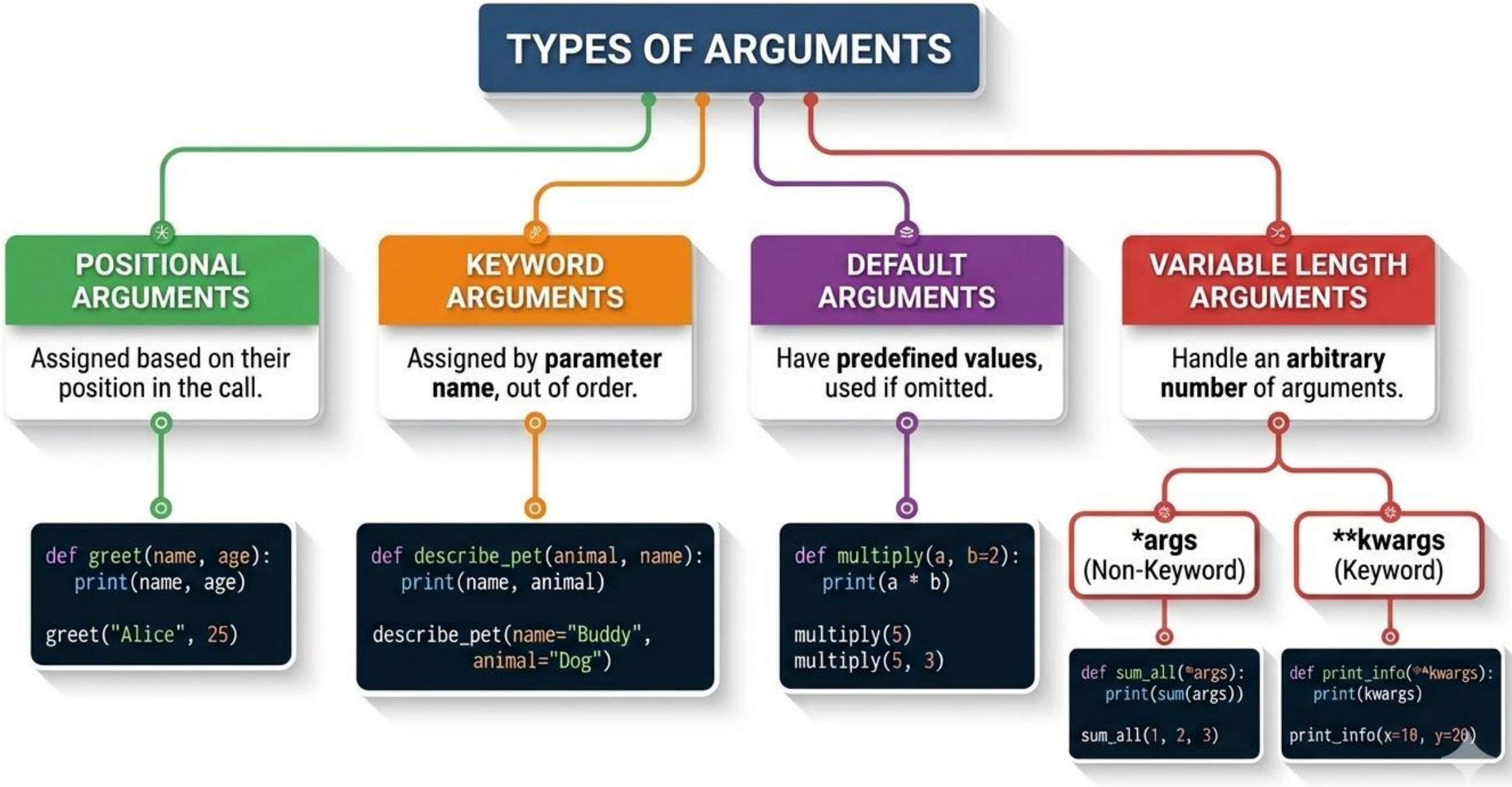
```
def function_name(parameters):
    """Docstring"""
    # body of the function
    return expression
```

Function Call ⟶ **Arguments** ⟶ Function ⟶ Result

Example: Even or Odd

```
def evenOdd(x):
    if (x % 2 == 0):
        return "Even"
    else:
        return "Odd"

print(evenOdd(16))
print(evenOdd(7))
```

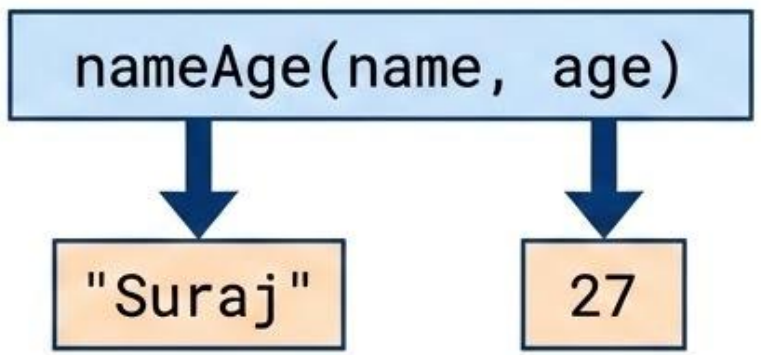
```
Even
Odd
```


TYPES OF ARGUMENTS
POSITIONAL ARGUMENTS
Assigned based on their position in the call.
def greet(name, age):
print(name, age)
greet("Alice", 25)
KEYWORD ARGUMENTS
Assigned by parameter name, out of order.
def describe_pet(animal, name):
print(name, animal)
describe_pet(name="Buddy",
animal="Dog")
DEFAULT ARGUMENTS
Have predefined values, used if omitted.
def multiply(a, b=2):
print(a * b)
multiply(5)
multiply(5, 3)
VARIABLE LENGTH ARGUMENTS
Handle an arbitrary number of arguments.
*args (Non-Keyword)
def sum_all(*args):
print(sum(args))
sum_all(1, 2, 3)
**kwargs (Keyword)
def print_info(**kwargs):
print(kwargs)
print_info(x=10, y=20)

Positional Arguments in Python

What are Positional Arguments?

- Values are assigned based on their **position**
- **Order** of arguments must match parameters
- Changing **order** changes the output
- Important for correct function execution



Values mapped by position

Example

```
def nameAge(name, age):
    print("Hi, I am", name)
    print("My age is", age)

print("Case-1:")
nameAge("Suraj", 27)

print("\nCase-2:")
nameAge(27, "Suraj")
```

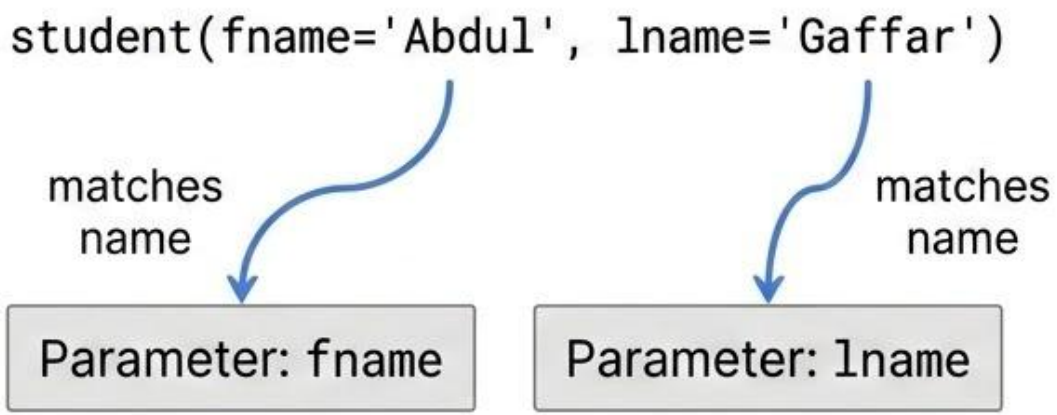
Output

```
Case-1:
Hi, I am Suraj
My age is 27

Case-2:
Hi, I am 27
My age is Suraj
```

Keyword Arguments in Python

What are Keyword Arguments?

- Values are passed using **parameter names**
- **Order** of arguments **does not matter**
- Improves clarity and readability
- Reduces errors in function calls



Example

```
def student(fname, lname):
    print(fname, lname)

student(fname='Abdul', lname='Gaffar')
student(lname='Gaffar', fname='Abdul')
```

Output:

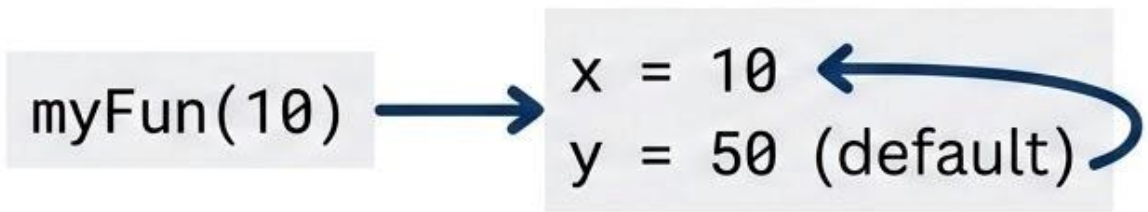
```
Abdul Gaffar
Abdul Gaffar
```

Same output even when order is changed

Default Arguments in Python

What are Default Arguments?

- A **default argument** has a predefined value
- Used when no value is provided during function call
- Makes parameters optional
- Helps avoid errors and simplifies function usage



Example

```
def myFun(x, y=50):
    print("x:", x)
    print("y:", y)

myFun(10)
```

```
Output:
x: 10
y: 50
```

y takes default value since it is not passed

Arbitrary Arguments in Python

1. What are Arbitrary Arguments?

- Allow a function to accept a variable number of inputs
- Useful when the number of arguments is unknown
- Makes function design more flexible

2. Types

*args

- Collects extra positional arguments
- Stored as a tuple

**kwargs

- Collects extra keyword arguments
- Stored as a dictionary

3. Purpose

Makes functions more flexible

Allows handling multiple inputs dynamically

***args** ⟶ (value1, value2, value3)

****kwargs** ⟶ {key1: value1, key2: value2}

Non-Keyword Arguments – *args in Python

What is *args?

- Allows a function to accept any number of positional arguments
- Arguments are passed without parameter names
- All values are stored as a **tuple**
- Useful when number of inputs is unknown

myFun('Hello','Welcome','to','Python')

args = ('Hello','Welcome','to','Python')

Examples

Example 1: Printing Values

```
def myFun(*argv):
    for arg in argv:
        print(arg)

myFun('Hello', 'Welcome', 'to', 'Python')
```

```
Hello
Welcome
to
Python
```

Example 2: Multiplication

```
def multiply(*args):
    result = 1
    for num in args:
        result *= num
    return result

print(multiply(2, 3, 4))
```

```
24
```

"***args** enables handling multiple values dynamically"

Keyword Arguments – **kwargs in Python

What is **kwargs?

- Allows a function to accept any number of **keyword arguments**
- Arguments are passed in the form key = value
- Values are stored in a **dictionary**
- Useful for handling named inputs dynamically

```
fun(s1='Python', s2='is', s3='Awesome')
```

```
kwargs = {s1: 'Python', s2: 'is', s3: 'Awesome'}
```

Examples

Example 1: Using kwargs

```
def fun(**kwargs):
    for k, val in kwargs.items():
        print(k, "=", val)

fun(s1='Python', s2='is', s3='Awesome')
```

Output:

```
s1 = Python
s2 = is
s3 = Awesome
```

Example 2: Using Both *args and kwargs

```
def student_info(*args, **kwargs):
    print("Subjects:", args)
    print("Details:", kwargs)

student_info("Math", "Science", "English",
             Name="Alice", Age=20, City="New York")
```

Output:

```
Subjects: ('Math', 'Science', 'English')
Details: {'Name': 'Alice', 'Age': 20, 'City': 'New York'}
```

***args** handles positional values, ****kwargs** handles named values

Anonymous Functions (Lambda) in Python

What are Anonymous Functions?

- Functions without a name
- Created using the **lambda** keyword
- Used for short, simple operations
- Typically written in a single line
- Useful when a function is needed temporarily

`def` → **Named** Function

`lambda` → **Anonymous** Function

Example

```
def cube(x):
    return x*x*x  # without lambda

cube_l = lambda x: x*x*x  # with lambda

print(cube(7))
print(cube_l(7))
```

Output

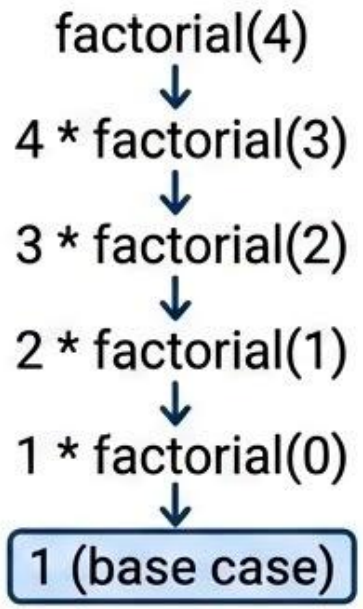
```
343
343
```

Both functions give the same result, but **lambda** is shorter

Recursive Functions in Python

What is Recursion?

- A function that calls itself to solve a problem
- Breaks a problem into smaller subproblems
- Commonly used in mathematical and divide-and-conquer problems
- Must include a **base case** to stop recursion
- Without a base case, it leads to infinite recursion



Example: Factorial

```python
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(4))
```

Output:
24

"Function keeps calling itself until **base case** (n == 0) is reached"

Core Concept of Recursion

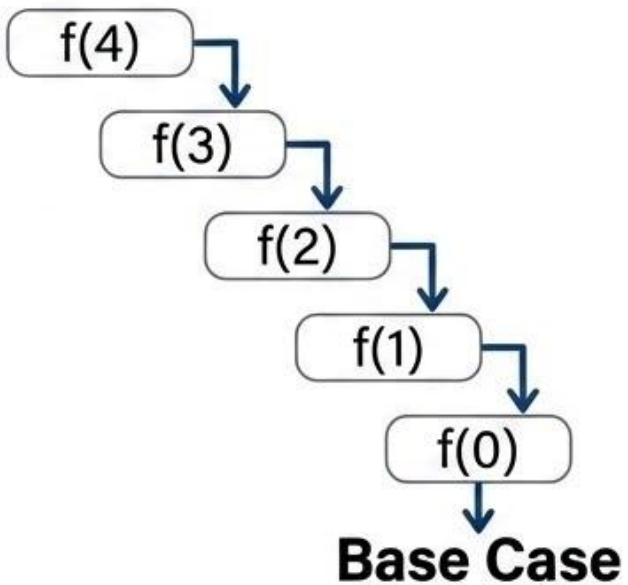
Core Concept

Recursion is a technique where a function solves a problem by calling itself on smaller inputs until it reaches a base case.

Real-Life Analogy

- You have a big task
- You give a smaller part to yourself again
- Keep reducing the work step by step
- When the task becomes very small → solve it directly

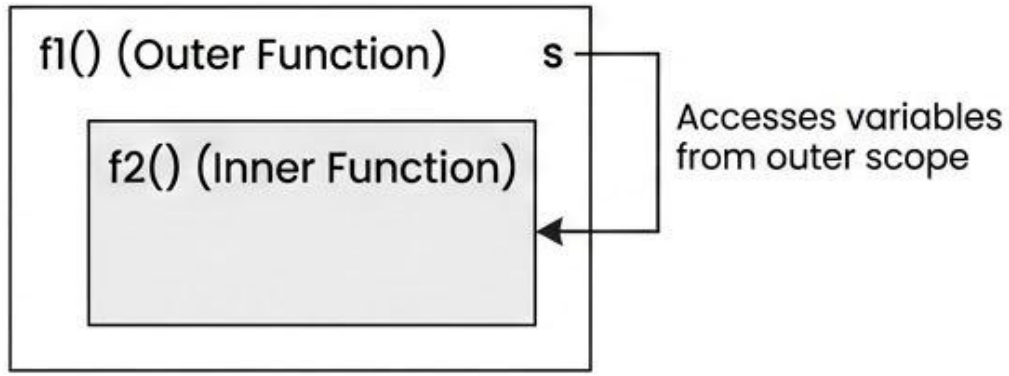
Recursive Process



Function within Functions (Nested Functions) in Python

What are Nested Functions?

- A function defined inside another function
- Also called an **inner** or **nested function**
- Can access variables from the outer (enclosing) function
- Helps organize and protect logic
- Improves code structure and readability



Example

```
def f1():
    s = 'I love GeeksforGeeks'

    def f2():
        print(s)

    f2()

f1()
```

Output:

```
I love GeeksforGeeks
```

Note: Inner function can access outer function variables (closure concept)

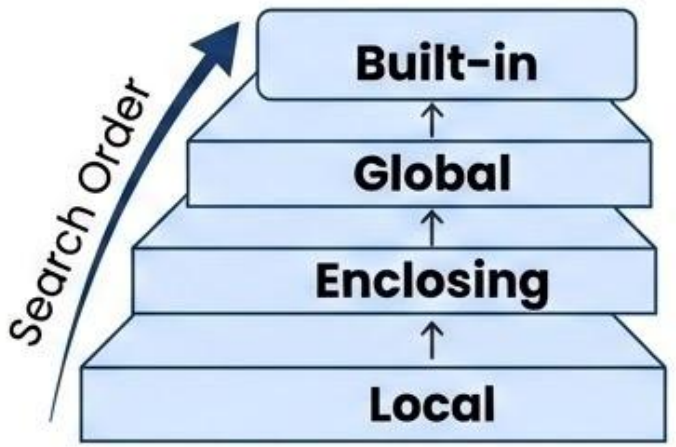
Scope of Variables in Python (LEGB Rule)

1. What is Variable Scope?

- Scope defines where a variable can be accessed
- Determines the visibility of variables in a program
- Python follows a specific order to search for variables

2. LEGB Rule

L → **Local** (inside a function)
E → **Enclosing** (inside outer function)
G → **Global** (entire script)
B → **Built-in** (Python reserved names)



3. Key Idea

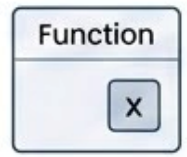
Python searches variables in this order:
Local → Enclosing → Global → Built-in

Types of Variable Scope in Python

1. **Local** Variables

- Defined inside a function
- Accessible only within that function

```
def test():
    x = 10
    print(x)

test()
```



2. **Global** Variables

- Defined **outside** all functions
- Accessible throughout the program

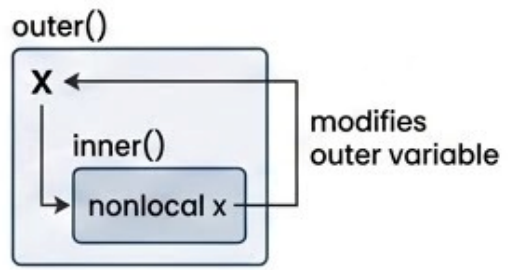
```
x = 20

def show():
    print(x)

show()
```

3. **Nonlocal** Variables

- Used in **nested** functions
- Refers to variables in the outer function
- Allows **modification** of outer function variable

```
def outer():
    x = 5
    def inner():
        nonlocal x
        x = 10
    inner()
    print(x)

outer()
```

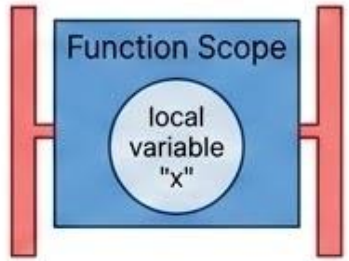
Output: 10



Modifying Variable Scope with Keywords

Default Behavior

- Variables inside functions are local by default
- They cannot modify variables outside the function unless specified

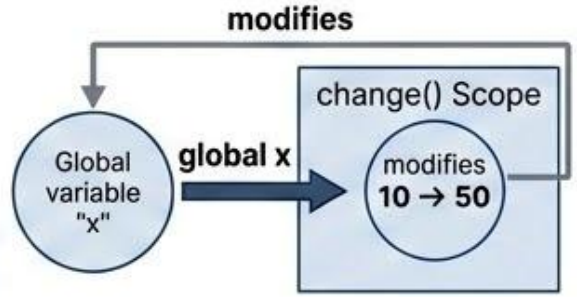


1. **global** Keyword

- Used to modify global variables inside a function
- Allows access to variables defined outside the function

```
x = 10

def change():
    global x
    x = 50

change()
print(x)
```

Output
50



2. **nonlocal** Keyword

- Used to modify variables in the enclosing (outer) function
- Works only with nested functions

```
def outer():
    x = 5
    def inner():
        nonlocal x
        x = 15
    inner()
    print(x)

outer()
```

Output
15